INSTITUT SUPERIEUR D'INFORMATIQUE

ET DES TECHNIQUES DE COMMUNICATION – HAMMAM SOUSSE

المعهد العالي للإعلامية وتكنولوجيات الاتصال بسوسة

End-of-Studies Internship Report

Presented for obtaining the Bachelor degree in Computer Science
Spécialité : Informatique and Multimédia

CyberScope

Web Application Security Auditing System
Detecting and Analyzing Vulnerabilities
Through Intelligent Automation

ELABORATED BY
MOHAMED GUIZANI

Academic Supervisor: M.Mohamed Amine Ben Amar

Professional Supervisor: M.Yassine Lasoued

Host Company

School Year : 2025/2026

Dedications

First and foremost, I thank **Allah** for choosing me among thousands to face this trial, which has made me more patient, wiser, and stronger. Despite the back injury that has challenged me for more than a year and a half, during which I could barely sit to work, all praise be to Allah for His guidance and strength.

To my beloved family, who replaced my physical pain with their unwavering support, love, and care:

To **Anissa, my mother**, whose devotion knew no bounds—checking on my condition and illness every single day throughout this difficult period. Your constant care has been my healing.

To **Hassen, my father**, who dedicated himself entirely to ensuring my comfort, doing everything possible to ease my burden. Your sacrifices have not gone unnoticed.

To **Iskander**, who became my backbone and eternal supporter when my own failed me. You carried me through the darkest moments.

To **Sabri**, from whom I drew power and strength when mine was depleted. Your presence gave me energy to continue.

To **Naktel**, the youngest among us, who brought joy and happiness into days filled with pain. Your light brightened my path.

Every line of this report was written through immense pain and is dedicated to all of you. Your love transformed my struggle into strength, and your faith in me made this achievement possible.

Alhamdulillahi rabbil alameen.

Acknowledgments

I would like to express my deepest gratitude to all those who have contributed to the completion of this thesis.

First and foremost, I extend my heartfelt appreciation to **M. Yassine Lassoed**, my professional supervisor, who believed in me and in this project from the very beginning. Despite the challenges faced during this internship, he provided me with essential guidance and constant support, enabling me to bring this project to life. His encouragement and trust in my abilities have been invaluable throughout this journey.

I would also like to sincerely thank **M. Mohamed Amine Ben Amar**, my academic supervisor, for his continuous support and valuable advice. Throughout this challenging period, he offered insightful feedback, motivation, and understanding, which greatly contributed to the successful completion of this work.

I am equally grateful to all the individuals who supported me, directly or indirectly, during the realization of this project. Their encouragement and assistance played a significant role in achieving this accomplishment.

Thank you for making this achievement possible.

Table of Contents

List of Abbreviations

API	Application Programming Interface
CSP	Content Security Policy
JWT	JSON Web Token
SPA	Single Page Application
SQLi	SQL Injection
XSS	Cross-Site Scripting
OWASP	Open Worldwide Application Security Project
UI	User Interface
DB	Database
HTTP	Hypertext Transfer Protocol

List of Figures

List of Tables

Introduction

In recent years, the rapid evolution of web technologies has significantly transformed the development and deployment of modern applications. Web applications have become increasingly dynamic, interactive, and complex, relying heavily on client-side rendering and continuous communication with backend services. While these advancements have enhanced user experience, they have also expanded the attack surface and introduced new security challenges.

Web application security has become a critical concern for organizations. Vulnerabilities such as SQL injection, cross-site scripting (XSS), and authentication flaws remain among the most exploited attack vectors, as highlighted by the OWASP Top 10. These vulnerabilities can lead to severe consequences, including data breaches, financial losses, and reputational damage.

To mitigate these risks, various security scanning tools have been developed, including solutions such as Nikto, OWASP ZAP, and Burp Suite. Although these tools provide valuable capabilities, they exhibit significant limitations. In particular, traditional scanners struggle to effectively analyze modern Single Page Applications (SPAs), where content is dynamically generated through JavaScript. Furthermore, they often produce redundant or low-priority findings, making vulnerability analysis more complex and less efficient.

In this context, there is a growing need for more intelligent and adaptive security auditing systems capable of handling modern web architectures while improving detection accuracy and result relevance.

To address these challenges, this project introduces **CyberScope**, an automated web application security auditing system designed to detect and analyze vulnerabilities through intelligent orchestration and advanced automation techniques. The proposed system integrates multiple security tools within a unified pipeline and enhances their capabilities through a browser-based crawling agent capable of exploring dynamic applications. In addition, an AI-driven correlation engine is employed to normalize, deduplicate, and prioritize detected vulnerabilities, providing more meaningful and actionable insights.

The objective of this work is to design and implement a comprehensive and efficient security auditing platform that improves vulnerability detection coverage and adapts to the complexity of modern web environments.

Chapter 1

General Presentation of the Project

1.1 Introduction

This chapter focuses on the general context of the project. It presents the background of the work, the security challenges faced by modern web applications, the limitations of existing solutions, and the motivation behind the proposed system, **CyberScope**.

1.2 Presentation of the Host Organization

Ernst & Young (EY) [1] is one of the world's leading professional services firms, recognized globally for its expertise in assurance, consulting, strategy, tax, and transaction services. Founded in 1989 through the merger of Ernst & Whinney and Arthur Young, EY operates in more than 150 countries and employs hundreds of thousands of professionals worldwide.

In Tunisia, EY plays a key role in supporting organizations in their digital transformation and technological innovation. The firm assists both public and private institutions in addressing complex business challenges by providing tailored consulting services and advanced technical solutions.

EY Tunisia offers a wide range of services, including IT consulting, cybersecurity, risk management, and digital innovation. In particular, the firm contributes to strengthening information systems security by identifying vulnerabilities and implementing robust protection mechanisms.

Through its global network and local expertise, EY promotes best practices in cybersecurity and technological development. Its commitment to quality and innovation makes it an ideal environment for the development of advanced solutions such as automated web security auditing systems.

1.2.1 EY at the International Level

At the international level, EY operates as a global professional services network that supports organizations across multiple industries. Its services cover assurance, consulting, tax, strategy, and transaction advisory. Through this worldwide presence, EY assists clients in improving their operational performance, managing risks, and adopting innovative technological solutions.

In the context of digital transformation, EY places strong emphasis on technology, data protection, governance, and cybersecurity. The firm helps organizations address emerging digital risks and adapt to increasingly complex regulatory and technological environments.

1.2.2 EY Tunisia and the Cybersecurity Division

EY Tunisia contributes to the digital transformation of Tunisian organizations by providing consulting and technology-oriented services. Its activities include IT consulting, risk management, cybersecurity, audit, and support for innovation projects.

The cybersecurity division focuses on helping organizations identify weaknesses in their information systems, assess risks, and improve their security posture. This includes activities such as vulnerability assessment, security auditing, governance support, and the implementation of protection strategies. This professional context is directly aligned with the objective of this project, which aims to design an automated and intelligent web application security auditing system.

Figure 1.1: EY Tunisia

1.3 Project Context and Problem Statement

With the increasing reliance on web applications in critical domains such as finance, e-commerce, and healthcare, ensuring their security has become a major challenge. Modern applications are more dynamic and complex, which significantly increases their exposure to various cyber threats such as SQL injection, cross-site scripting, and authentication vulnerabilities.

Despite the availability of several security tools, existing solutions often fail to effectively analyze modern web applications, especially those based on dynamic architectures. These tools may miss critical vulnerabilities, generate redundant results, or require significant manual intervention, which reduces their efficiency and reliability.

In this context, the main problem addressed in this project is the lack of an intelligent and automated solution capable of efficiently auditing modern web applications while improving vulnerability detection accuracy and reducing false positives.

The objective of this project is therefore to design and develop **CyberScope**, an advanced web application security auditing system that enhances vulnerability detection through intelligent orchestration, automation, and improved analysis of security

findings.

1.4 Overview of Web Application Security

Web application security refers to the set of measures and practices implemented to protect web applications from vulnerabilities and cyber threats. As web applications become increasingly central to business operations and data management, ensuring their security is essential to preserve the confidentiality, integrity, and availability of information.

Modern web applications are exposed to a wide range of attacks that exploit weaknesses in their design, implementation, or configuration. These attacks may target input validation, authentication mechanisms, session management, or communication between client and server. As a result, a single vulnerability can compromise the entire system and lead to severe consequences such as data breaches, financial loss, or reputational damage.

To mitigate these risks, security must be integrated throughout the entire development lifecycle. This includes secure coding practices, regular vulnerability assessments, and the use of automated security tools capable of identifying potential weaknesses before they are exploited.

In this context, standards such as the OWASP Top 10 provide valuable guidance by identifying the most critical web application security risks [2]. These references help developers and security professionals prioritize their efforts and implement effective protection strategies.

1.4.1 OWASP Top 10 and Modern Threat Landscape

The OWASP Top 10 is a widely recognized standard that identifies the most critical security risks affecting web applications [2]. It provides a prioritized list of vulnerabilities such as injection attacks, broken authentication, sensitive data exposure, and cross-site scripting.

These vulnerabilities represent the most common attack vectors exploited by attackers in real-world scenarios. As web technologies evolve, the threat landscape continues to expand, with modern applications introducing new risks related to client-side execution, APIs, and complex system interactions.

By highlighting the most significant threats, the OWASP Top 10 helps developers and security professionals focus on the most impactful vulnerabilities and adopt effective mitigation strategies.

Table 1.1: OWASP Top 10 Web Application Security Risks

ID	Category
A01	Broken Access Control
A02	Cryptographic Failures
A03	Injection
A04	Insecure Design
A05	Security Misconfiguration
A06	Vulnerable and Outdated Components
A07	Identification and Authentication Failures
A08	Software and Data Integrity Failures
A09	Security Logging and Monitoring Failures
A10	Server-Side Request Forgery

1.4.2 Statistics and Economic Impact of Web Vulnerabilities

The impact of web application vulnerabilities is not limited to technical compromise. Security incidents can also lead to financial losses, service disruption, legal consequences, and reputational damage. According to IBM's Cost of a Data Breach Report 2024, the global average cost of a data breach reached USD 4.88 million, representing a significant increase compared to the previous year [3].

In addition, the Verizon Data Breach Investigations Report highlights the importance of vulnerability exploitation as a major attack vector in real-world incidents [4]. This shows that unpatched weaknesses and exposed web services remain attractive targets for attackers.

These findings confirm the need for proactive and continuous web application security auditing. Automated solutions can help organizations identify vulnerabilities earlier, reduce remediation delays, and limit the potential impact of security incidents.

1.4.3 Limitations of Traditional Vulnerability Scanners

Traditional vulnerability scanners are widely used to detect security flaws in web applications through automated analysis and predefined signatures. While these tools are effective for identifying known vulnerabilities, they present several limitations when applied to modern web environments.

In particular, they often struggle to analyze dynamic applications, especially those based on client-side rendering and asynchronous communication. As a result, important attack surfaces may remain unexplored. Additionally, these tools may generate a high number of false positives and redundant findings, which complicates the analysis process and reduces their overall efficiency.

These limitations highlight the need for more advanced and intelligent approaches to web application security auditing.

1.5 Study of Existing Solutions

Several tools have been developed to identify vulnerabilities in web applications and assist security professionals in detecting potential threats. These tools provide different approaches to vulnerability scanning, ranging from simple automated checks to advanced interactive analysis. Among the most widely used solutions are Nikto, OWASP ZAP, and Burp Suite.

1.5.1 Nikto

Nikto is an open-source web server scanner designed to detect known vulnerabilities, outdated software, and misconfigurations in web servers [5]. It performs comprehensive checks against a large database of known issues and is commonly used for quick security assessments.

However, Nikto is limited to server-side analysis and does not effectively handle modern dynamic web applications, which reduces its applicability in complex environments.

Figure 1.2: Nikto

1.5.2 OWASP ZAP

OWASP ZAP is a widely used open-source security testing tool that provides automated scanning as well as manual testing capabilities [6]. It is capable of detecting a variety of vulnerabilities such as injection flaws and security misconfigurations.

Despite its flexibility, ZAP may require proper configuration and user expertise to achieve optimal results, and it can produce a significant number of false positives in complex applications.

Figure 1.3: OWASP ZAP

1.5.3 Burp Suite

Burp Suite is a professional web security testing platform offering a comprehensive set of tools for vulnerability detection and exploitation [7]. It allows detailed analysis of

HTTP requests and responses and supports advanced manual testing techniques.

While powerful, Burp Suite is more complex to use and often requires significant manual intervention, which can make the testing process time-consuming.

Figure 1.4: Burp Suite

1.5.4 AI-Driven Security Testing Solutions

Recent security testing platforms have started integrating artificial intelligence and advanced automation in order to improve vulnerability detection, prioritization, and remediation guidance. Examples include solutions such as StackHawk, Detectify, and Snyk [8, 9, 10].

StackHawk focuses on modern Dynamic Application Security Testing and integration into CI/CD pipelines, allowing development teams to detect vulnerabilities earlier in the software lifecycle. Detectify provides attack surface monitoring and application scanning capabilities, helping organizations continuously discover and assess exposed assets. Snyk focuses on identifying vulnerabilities in code, dependencies, containers, and infrastructure-as-code.

Although these solutions introduce valuable automation and intelligence, they do not fully address the specific objective of this project: building a local, customizable, multi-tool orchestration platform capable of combining dynamic crawling, result deduplication, AI-based interpretation, and professional reporting within a unified workflow.

1.5.5 Comparative Analysis

To better evaluate the strengths and limitations of the studied tools, a comparative analysis is presented below. Although Nikto, OWASP ZAP, and Burp Suite are widely used in web application security testing, each of them addresses only part of the overall auditing process.

Nikto is well suited for rapid server-side checks, OWASP ZAP provides a balanced approach between automation and usability, and Burp Suite offers advanced manual testing capabilities. However, none of these tools fully combines intelligent automation, deep dynamic exploration, and effective reduction of noisy findings.

Table 1.2: Comparative analysis of existing solutions

Criteria	**Nikto**	**OWASP ZAP**	**Burp Suite**	**AI-driven Solutions**
Type	Open-source server scanner	Open-source proxy and scanner	Professional security testing suite	Automated security platforms
Main approach	Signature-based scanning	Automated and manual testing	Advanced manual and automated testing	Continuous testing and prioritization
Dynamic application support	Limited	Partial	Good	Good depending on platform
AI-assisted analysis	No	Limited	Limited	Yes
False positives	High	Medium	Low to medium	Reduced through prioritization
Manual testing support	No	Yes	Advanced	Limited to platform features
Automation level	High	High	Medium	High
Local customization	High	High	Medium	Limited in cloud-based platforms
Best suited for	Quick server checks	General web security testing	Deep vulnerability analysis	Continuous security monitoring

This comparison shows that existing tools provide useful but incomplete capabilities. Their limitations in terms of automation, adaptability, dynamic analysis, and contextual interpretation justify the need for a more integrated and intelligent solution such as **CyberScope**.

1.6 Critique of Existing Solutions

Although existing tools such as Nikto, OWASP ZAP, and Burp Suite provide valuable support for vulnerability detection, they still present several limitations when applied to modern web applications.

These limitations are mainly related to the lack of support for dynamic applications, the generation of false positives, and the need for manual intervention. In addition, the absence of intelligent correlation between detected vulnerabilities reduces

the clarity and efficiency of the analysis process.

Table 1.3: Synthesis of limitations of existing solutions

Limitation	**Description**	**Impact**
Limited support for dynamic applications	Difficulty handling JavaScript-based applications	Incomplete vulnerability detection
High false positives	Redundant or irrelevant findings	Time-consuming analysis
Lack of intelligent correlation	No prioritization of vulnerabilities	Reduced clarity of results
Manual intervention required	Dependence on user expertise	Low scalability
Fragmented tool usage	Multiple tools used separately	Complex workflow

These limitations highlight the need for a more advanced solution capable of combining automation, accuracy, and adaptability.

1.7 Proposed Solution: CyberScope

To overcome the limitations identified in existing solutions, this project proposes **CyberScope**, an intelligent web application security auditing system.

CyberScope is based on a unified architecture that integrates multiple security tools into a single automated pipeline. It introduces a browser-based crawling mechanism capable of exploring modern dynamic applications and interacting with client-side components.

In addition, the system incorporates an intelligent analysis layer responsible for normalizing, deduplicating, and prioritizing vulnerabilities. This significantly reduces noise and improves the relevance of the results.

By combining automation, dynamic exploration, and intelligent processing, CyberScope aims to provide a more efficient and scalable approach to web application security auditing.

Figure 1.5: CyberScope Logo

1.7.1 Vision and Objectives

The vision of **CyberScope** is to provide an enterprise-oriented web application security auditing platform that helps security analysts detect, understand, and prioritize vulnerabilities more efficiently. To realize this vision, the platform automates the coordinated execution of multiple open-source security tools within a unified workflow, improves coverage of modern dynamic applications through browser-driven exploration, and reduces duplicated or noisy findings via normalization and deduplication. An AI-assisted analysis layer produces clearer vulnerability interpretations and risk prioritization, and results are delivered through a professional dashboard with automated, structured audit reports for stakeholders.

1.7.2 Contributions and Added Value Compared to Existing Solutions

CyberScope does not aim to replace existing scanners. Instead, it acts as an intelligent orchestration and analysis layer that enhances their effectiveness by combining their outputs and presenting them in a more structured and actionable form.

Table 1.4: Added value of CyberScope compared to existing solutions

Aspect	**Existing Solutions**	**CyberScope Contribution**
Tool execution	Tools are generally used separately	Unified orchestration of multiple security tools
Dynamic analysis	Limited or partial support for modern applications	BrowserAgent for exploring dynamic interfaces and hidden endpoints
Result processing	Raw outputs or tool-specific formats	Normalized and deduplicated vulnerability findings
Risk interpretation	Mostly technical results requiring manual analysis	AI-assisted interpretation and prioritized remediation guidance
Visualization	Basic reports or complex expert interfaces	Structured dashboard and clearer vulnerability presentation
Reporting	Often manual or tool-dependent	Automated generation of professional audit reports

These contributions position CyberScope as an integrated auditing platform that improves automation, readability, and decision-making in web application security assessment.

1.8 Conclusion

This chapter introduced the general context of the project by highlighting the importance of web application security and the challenges posed by modern dynamic applications. It also presented a study of existing solutions and identified their main limitations.

Based on this analysis, the need for a more advanced and intelligent approach was established, leading to the proposal of **CyberScope**. The next chapter will focus on the analysis of the system requirements and the initialization of the Scrum process.

Chapter 2

System Analysis and Requirements Specification

2.1 Introduction

This chapter presents the analysis and specification of the **CyberScope** system. It introduces the adopted development methodology, identifies the main stakeholders, presents the development environment, and defines both functional and non-functional requirements. It also presents the use case diagram, the textual description of the main use cases, and the organization of the project through the product backlog and sprint planning.

2.2 Adopted Development Methodology

To ensure a flexible and progressive development process, the Scrum methodology was adopted for the realization of **CyberScope**. Scrum is an agile framework based on iterative development, incremental delivery, continuous feedback, and adaptation to evolving requirements.

2.2.1 Justification of the Choice of Scrum

The choice of Scrum is justified by the nature of the CyberScope project, which involves several heterogeneous components such as a frontend dashboard, a backend API, a Python-based scanning engine, external security tools, and AI-based analysis modules.

A traditional sequential development approach would make it difficult to validate each component progressively. Scrum allows the project to be divided into smaller increments, making it possible to test, validate, and improve each part of the system throughout development.

2.2.2 Adaptation of Scrum to an Individual Project

Although Scrum is typically applied in team-based environments, it was adapted to the context of an individual end-of-studies project. The student assumed multiple roles simultaneously, including development, planning, and coordination.

The academic and professional supervisors contributed as stakeholders by validating the objectives, monitoring progress, and providing feedback. This adaptation preserved the benefits of Scrum while accommodating the constraints of a single-developer project.

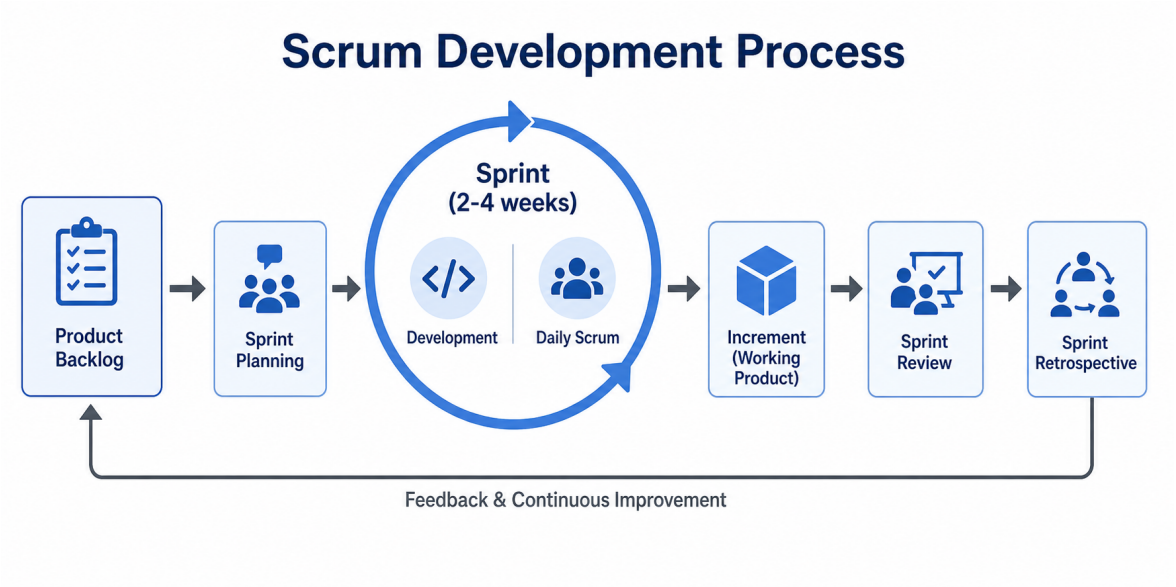

Figure 2.1: Scrum Development Process adapted to CyberScope

2.2.3 Organization of Sprints and Product Backlog

Table 2.1: Sprint Organization of CyberScope

Sprint	Main Objective	Deliverable	Duration
Sprint 1	Backend foundation and database setup	Core backend APIs and database structure	3 weeks
Sprint 2	Scanning engine and tool orchestration	Integration of tools and scanning pipeline	3 weeks
Sprint 3	Frontend dashboard development	User interface and result visualization	3 weeks
Sprint 4	Advanced features and validation	AI analysis, reporting, and testing	3 weeks

Backlog estimation methods

T-shirt sizing: For a faster, higher-level estimate during roadmap planning we applied the T-shirt sizing method (XS, S, M, L, XL). This method groups stories by relative size classes rather than numeric points.

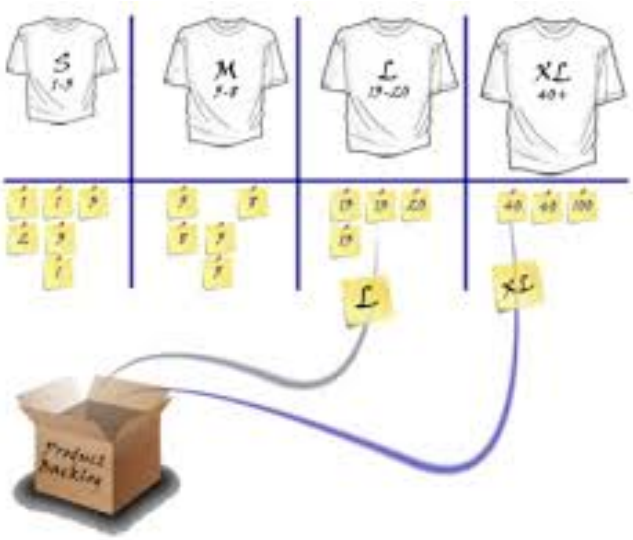

Figure 2.2: T-shirt sizing categories used for coarse backlog estimation

Table 2.2: Product Backlog of CyberScope

Sprint	Category	User Story	Priority	Complexity
Sprint 1	Auth	PB-1 — As a user, authenticate securely with JWT tokens	High	S
Sprint 2	Scanning	PB-2 — Submit a target URL and initiate a scan	High	L
Sprint 2	Orchestration	PB-3 — Orchestrate multiple security tools	High	L
Sprint 2	Crawling	PB-4 — BrowserAgent for dynamic applications	High	XL
Sprint 3	UI	PB-5 — View scan results on an intuitive dashboard	High	M
Sprint 3	UI	PB-6 — Filter and sort vulnerabilities	Medium	S
Sprint 2	Processing	PB-7 — Deduplicate and normalize findings	High	L
Sprint 4	Reporting	PB-8 — Generate comprehensive security reports	Medium	M
Sprint 4	Analysis	PB-9 — Analysis: AI-based analysis and risk assessment	Medium	XL
Sprint 3	History	PB-10 — View scan history and previous results	Low	XS

2.3 Stakeholder Identification

The development and use of **CyberScope** involve several stakeholders, each interacting with the system according to specific roles.

The primary stakeholder is the **Security Analyst**, who represents the main user of the platform. This actor submits target URLs, launches scans, and analyzes results through the dashboard.

The **System Administrator** is responsible for maintaining the system, managing configurations, and ensuring the availability of integrated tools.

Academic and professional supervisors act as indirect stakeholders. They validate the project objectives and ensure the quality of the delivered solution.

Finally, the **target application owner** represents an external stakeholder who benefits from the security analysis results.

2.4 System Overview

CyberScope is an intelligent web application security auditing platform designed to automate vulnerability detection and analysis. It integrates multiple security tools into a unified pipeline and provides advanced features such as dynamic crawling, result aggregation, and AI-driven interpretation.

The system is composed of three main components:

- A frontend dashboard for user interaction and visualization
- A backend orchestration layer managing scans and data
- A scanning engine responsible for executing security tools

2.5 Functional Requirements

The functional requirements define the core services provided by CyberScope. These functionalities describe how the system interacts with users and how it performs automated security analysis, from scan initiation to result reporting.

2.5.1 Scan Initiation and Target Submission

The system must allow users to submit a target web application through a valid URL and initiate a security scan. Prior to execution, the system must validate the input to ensure correctness and accessibility of the target.

Once validated, the system must create a scan instance, assign it a unique identifier, and trigger the scanning workflow. The user must be able to monitor the progress of the scan (e.g., pending, running, completed, failed) through the interface. Additionally, the system should support multiple concurrent scans and maintain a history of previous scans for later consultation.

2.5.2 Intelligent Tool Orchestration

CyberScope must coordinate the execution of multiple security tools within a unified and automated pipeline. The system should dynamically orchestrate tools such as

Nmap, Nuclei, Nikto, and SQLMap depending on the target characteristics.

The orchestration process must ensure efficient scheduling, avoid redundant executions, and support parallel processing when possible. It must also manage tool configuration, execution, and result collection transparently, without requiring manual user intervention.

2.5.3 Dynamic Application Crawling (BrowserAgent)

The system must include a browser-based crawling component, referred to as the **BrowserAgent**, capable of interacting with modern web applications.

This component must simulate real user behavior by executing JavaScript, navigating through dynamic interfaces, and capturing network requests. It enables the discovery of hidden endpoints, API calls, and parameters that are not accessible through traditional static scanning techniques.

This functionality is particularly important for analyzing Single Page Applications (SPAs) and dynamically generated content.

2.5.4 Vulnerability Detection and Analysis

The system must detect a wide range of web application vulnerabilities, including SQL injection, cross-site scripting (XSS), exposed services, insecure configurations, and outdated components.

CyberScope must process the outputs of integrated tools and transform them into structured vulnerability objects. Each vulnerability should include detailed information such as its type, severity level, affected endpoint, technical evidence, and potential impact.

This structured representation facilitates further analysis, prioritization, and decision-making.

2.5.5 Aggregation and Visualization of Results

The system must aggregate results collected from multiple tools and eliminate duplicate findings through normalization and deduplication mechanisms.

The processed results must be presented through an intuitive dashboard that allows users to explore vulnerabilities using filters, sorting options, and visual indicators. The interface should provide a clear overview of the security posture, including severity distribution, affected components, and scan statistics.

This improves readability and enables users to quickly identify and prioritize critical issues.

2.5.6 Report Generation

The system must generate comprehensive and structured reports summarizing the results of a security scan.

These reports should include target information, detected vulnerabilities, severity classification, risk assessment, and remediation recommendations. The reports must be exportable in a professional format (e.g., PDF) to facilitate communication with stakeholders and support auditing processes.

2.6 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the CyberScope system. They ensure that the platform is efficient, secure, reliable, and maintainable.

2.6.1 Performance and Scalability

The system must provide acceptable performance in terms of response time and processing efficiency. While vulnerability scans may be resource-intensive, user interactions with the platform must remain responsive.

The architecture should support scalability by allowing the distribution of system components (frontend, backend, scanning engine) to handle increasing workloads and larger target applications without significant performance degradation.

2.6.2 Security Requirements

CyberScope must implement strong security mechanisms to protect user data and scan results. Authentication and authorization must be enforced using secure approaches such as JSON Web Tokens (JWT).

Sensitive data must be protected against unauthorized access, and secure communication must be ensured between system components. Given the nature of the platform, maintaining confidentiality and integrity is essential.

2.6.3 Usability

The platform must provide an intuitive and user-friendly interface that simplifies interaction with complex security data.

The dashboard should present information in a clear and structured manner, allowing users to easily navigate between scans, analyze vulnerabilities, and access reports. Important elements such as severity levels and risk indicators should be highlighted to improve decision-making.

2.6.4 Reliability and Availability

The system must ensure stable and reliable operation during scan execution and data processing. It should handle failures gracefully, such as tool crashes or network interruptions, without compromising data integrity.

Appropriate logging and error-handling mechanisms must be implemented to support monitoring and debugging. The system should also maintain high availability

with minimal downtime.

2.6.5 Maintainability and Extensibility

CyberScope must be designed using a modular architecture that facilitates maintenance and future evolution.

The system should allow easy integration of new security tools, updates to existing components, and modifications without affecting the overall system stability. Clear separation between components (frontend, backend, scanning engine) enhances maintainability.

Extensibility is essential to adapt the platform to new vulnerabilities, technologies, and evolving security requirements.

2.7 Global Use Case Diagram

The use case diagram illustrates the main interactions between the user and the CyberScope system.

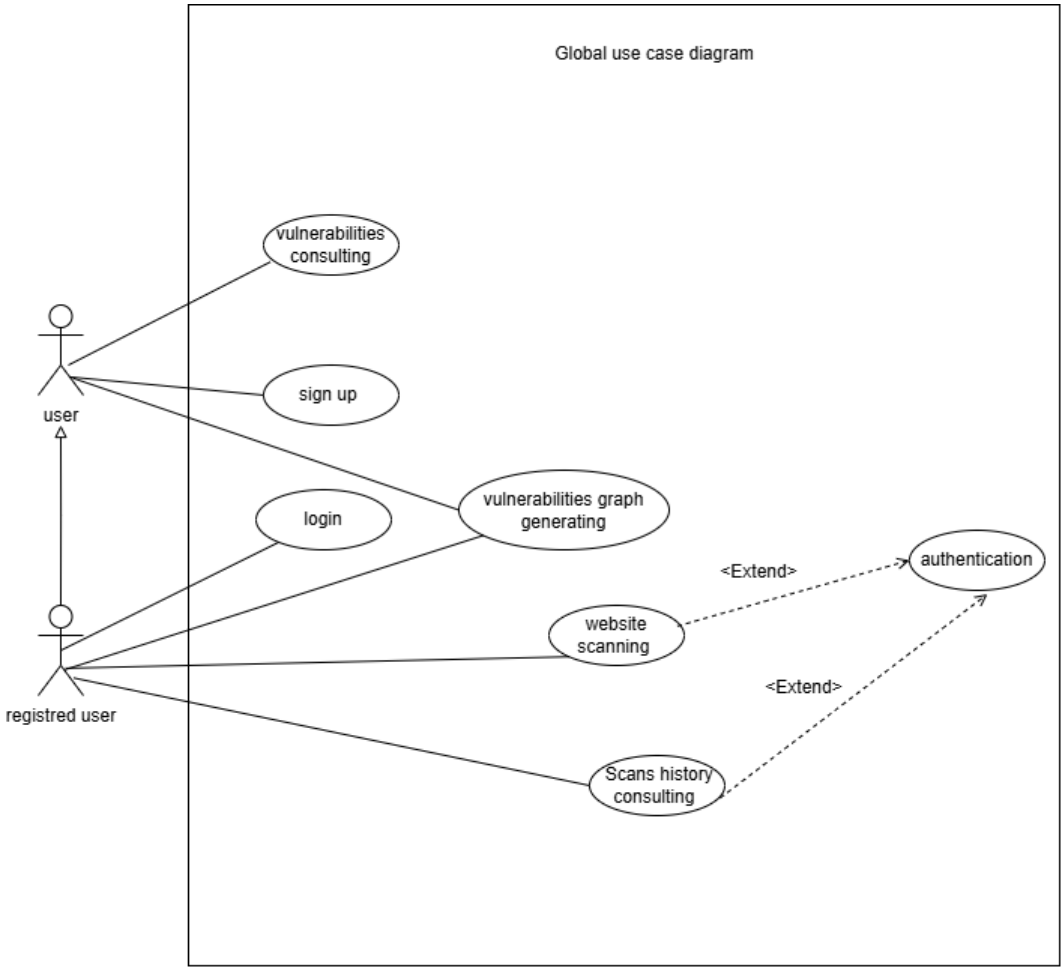

Figure 2.3: Global Use Case Diagram of CyberScope

2.7.1 Global Class Diagram

The class diagram presents the main structural components of CyberScope and the relationships between its core services, models, and orchestration layers.

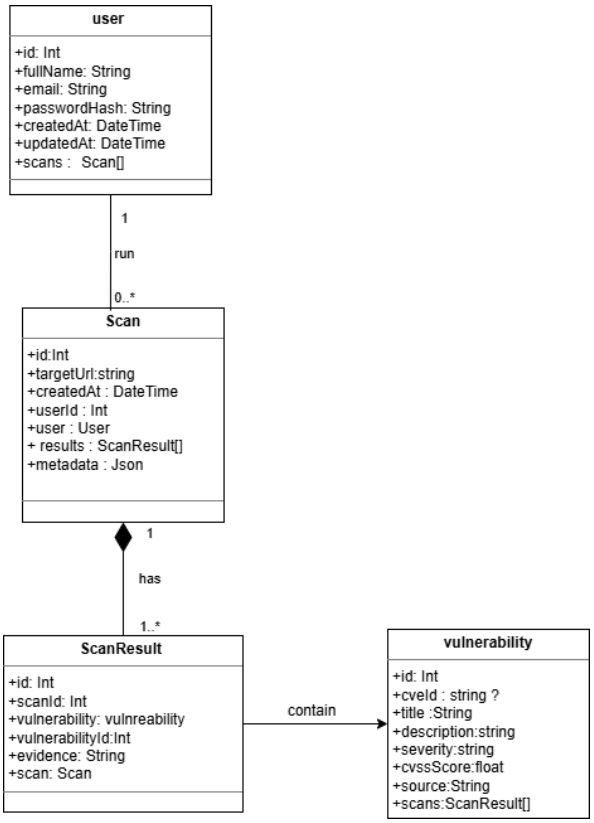

Figure 2.4: Global Class Diagram of CyberScope

2.8 Development Environment

This section presents the technical environment used to design, implement, and test **CyberScope**. It highlights the main programming languages, frameworks, tools, and software resources that supported the project.

2.8.1 Material Environment

The development of **CyberScope** was carried out on a Windows-based workstation. The main tools used in this environment were **Visual Studio Code**, which served as the primary code editor for development, debugging, and workspace management; **Git**, which was used to track changes and manage the source code history; and **Google Chrome / Chromium**, which was used for manual testing, dynamic debugging, and inspecting web application behavior.

2.8.2 Software Environment

The software environment of **CyberScope** is organized around three main layers: the backend API, the frontend interface, and the external intelligence and scanning tools.

On the backend, the system relies on **Node.js** as the runtime environment, **Express.js** for the RESTful API and routing logic, **Prisma** as the ORM used to interact with the PostgreSQL database and manage schema migrations, and **PostgreSQL** for persisting scan data, users, and results. On the frontend, **React** is used to build the interactive dashboard and components, while **Vite** provides the development server and fast hot-module reloading, and **TypeScript** improves code quality and reduces runtime errors.

The project also integrates external tools and services for scanning, orchestration, and AI-assisted analysis. **Python** is used to implement the scanning engine and the glue code for tool integrations, **Ollama** serves as the local AI engine for generating analysis prompts and assisting risk assessment, and **HexStrike AI** acts as the orchestration layer and automation platform for managing multiple security tools. In addition, **Chromium / Google Chrome** is used as the browser runtime for dynamic crawlers and headless scanning components.

2.9 Conclusion

This chapter presented the analysis and specification of the CyberScope system, including the adopted methodology, stakeholder identification, development environment, system overview, and requirements. These elements provide a solid foundation for the system design presented in the next chapter.

Chapter 3

Backend Foundation & Authentication System

3.1 Introduction

This sprint focuses on the authentication layer of the HexStrike platform and on the access control rules that protect the main application modules. The goal of this iteration is to provide a secure entry point to the system, manage user sessions with JWT, and restrict sensitive actions such as scan creation and scan history access to authenticated users only.

In practice, this sprint covers the login and registration flow, the guest session mechanism, protected routes on the frontend, and the backend middleware that validates every secured request.

3.1.1 Sprint Purpose

The purpose of this sprint is to secure the platform's entry points and session management to enable safe use of the scanning and analysis features. Concretely, the sprint delivers a robust authentication system, a guest mode for quick evaluation, and middleware that enforces access control for all sensitive API endpoints.

3.2 Sprint Backlog

The backlog of this sprint contains the following user stories:

As a visitor, I can create an account and log in to the platform.
As a user, I can start a temporary guest session without creating a persistent account.
As an authenticated user, I can access protected pages such as the scanner and scan history.
As a guest user, I cannot access protected resources or save scans in the database.
As the system, I can verify and refresh the current session by exposing the authenticated user profile.

Table 3.1: Sprint Backlog

3.3 Requirements Analysis

3.3.1 Use Case Diagram

The actors involved in this sprint are the visitor, the registered user, and the guest user. The visitor can register and log in, the registered user can access protected modules,

and the guest user can only use limited temporary access.

The following figure is reserved for the use case diagram of this sprint.

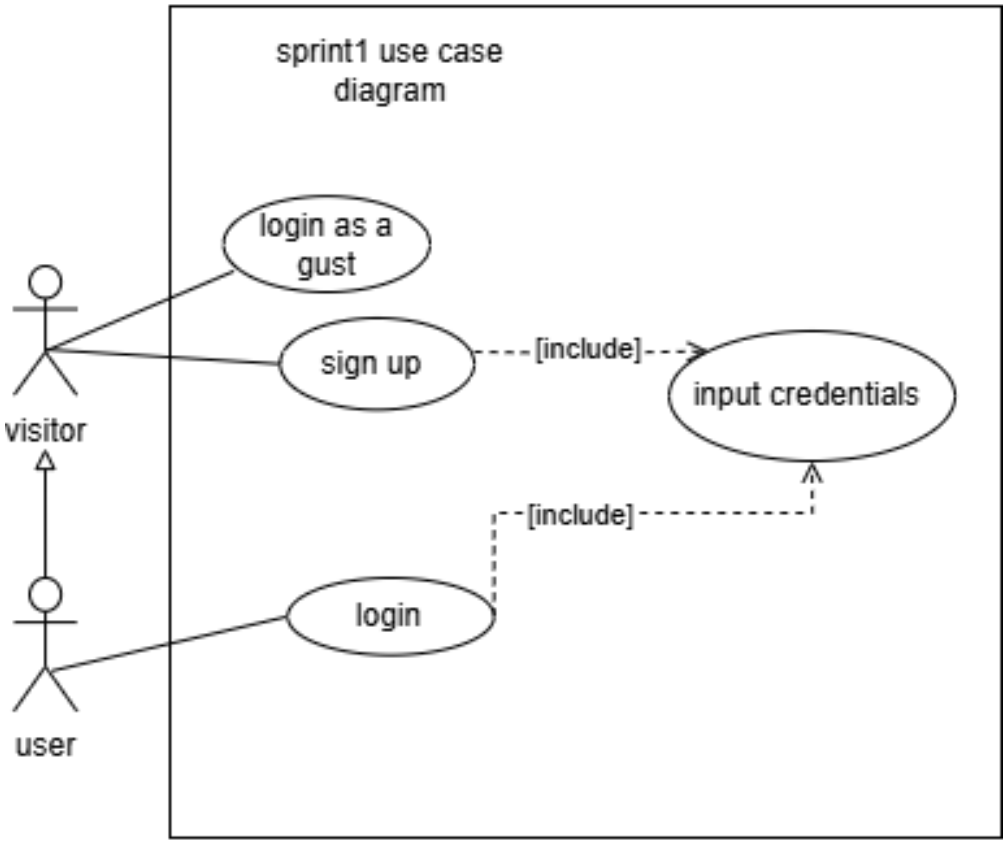

Figure 3.1: Use case diagram for authentication and access control

3.3.2 Use Case Description

The main use cases of this sprint are summarized in the following tables.

Use case	Register
Actor	Visitor
Description	The visitor creates a new account by providing a full name, an email address, and a password. The backend validates the input, hashes the password, stores the user in the database, and returns a JWT token.
Pre-condition	The visitor does not already have an active authenticated session.
Post-condition	The user account is created and the session becomes authenticated.

Table 3.2: Use case description: Register

Use case	Login
Actor	User
Description	The user provides valid credentials. The backend checks the password, generates a signed JWT token, and returns the authenticated profile.
Pre-condition	The user account already exists in the database.
Post-condition	The user is authenticated and can access protected pages.

Table 3.3: Use case description: Login

Use case	Guest login
Actor	Visitor
Description	The system creates a temporary guest session for quick access. This session is limited and does not allow scan persistence or access to scan history.
Pre-condition	The user is not connected with a registered account.
Post-condition	A temporary guest token is created and stored on the client side.

Table 3.4: Use case description: Guest login

3.4 Design

3.4.1 Class Diagram

The design of this sprint is based on the separation between frontend and backend responsibilities.

On the backend side, the authentication flow is handled by the Auth controller and service, the user data is stored in the Prisma User model, and the secured routes are protected by the authentication middleware. The server also applies rate limiting to login endpoints to reduce brute-force attempts.

On the frontend side, the AuthContext manages the session state, the Login and Register pages collect credentials, and the ProtectedRoute and PublicOnlyRoute components control access to the application pages.

The following figure is reserved for the refined class diagram of this sprint.

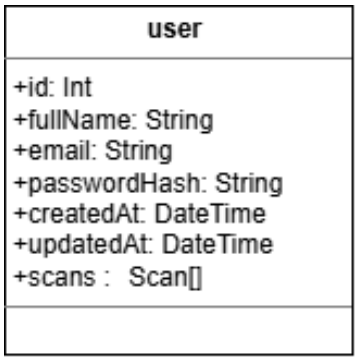

Figure 3.2: Class diagram for Sprint 1

3.4.2 Sequence Diagram

The main sequence of this sprint starts when the user submits login credentials from the frontend. The frontend sends the request to the backend, the backend verifies the credentials, and if they are valid, a JWT token is returned and stored locally by the client.

After authentication, every protected request includes the token in the authorization header. The backend middleware checks the token before allowing access to scan creation, scan history, and detailed scan reports. Guest sessions follow the same flow, but the backend blocks access to privileged operations.

The following figure is reserved for the sequence diagram of this sprint.

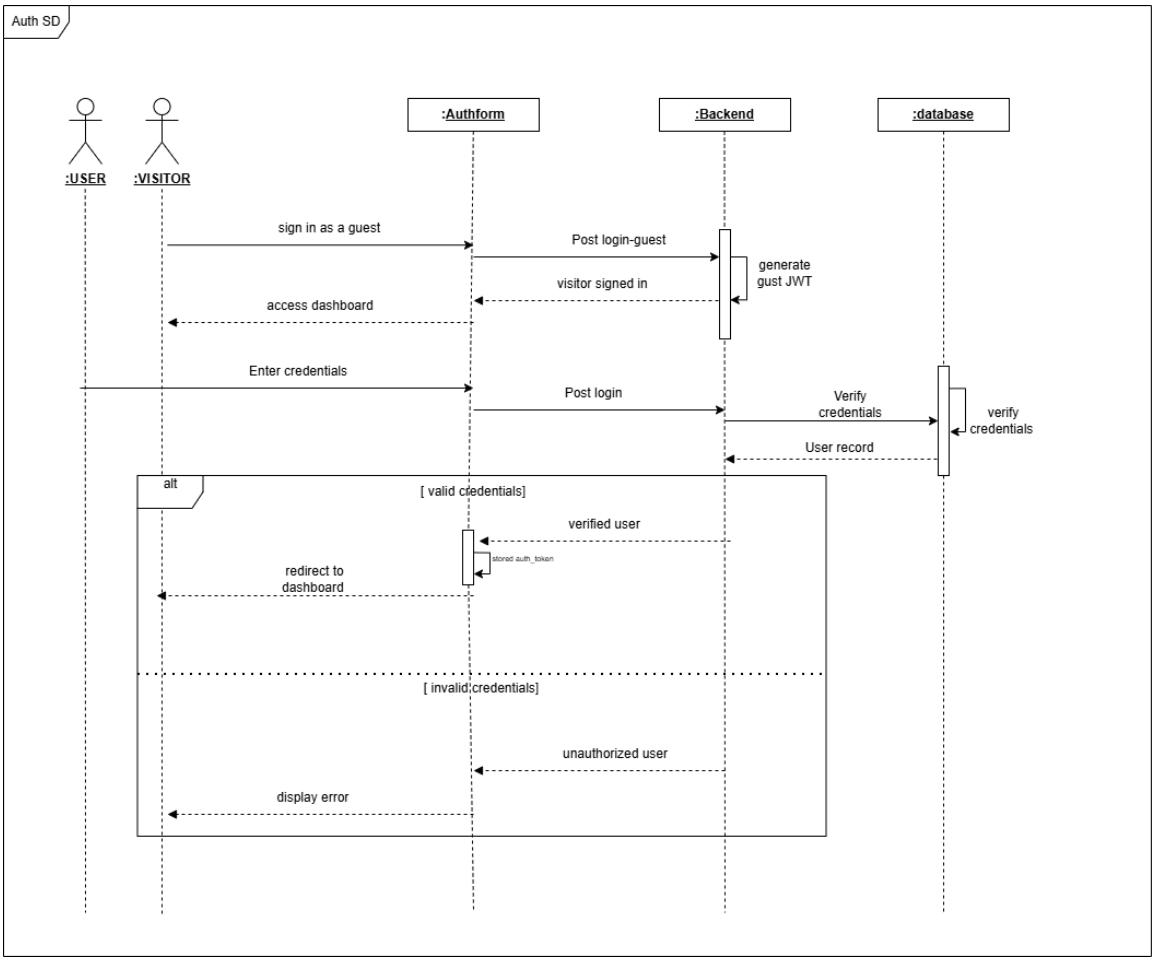

Figure 3.3: Sequence diagram for authentication and access control

3.5 Implementation

This sprint is implemented with an Express.js backend and a React frontend. The implementation of the sprint is illustrated through screenshots of the application interfaces.

On the backend, the main authentication endpoints are defined under `/api/auth`. The register and login endpoints create secured sessions, the guest endpoint creates a limited temporary session, and the `/me` endpoint returns the current authenticated user. The backend stores users in PostgreSQL through Prisma and uses bcrypt to hash passwords before saving them.

On the frontend, the Login page sends credentials to the backend, the AuthContext stores the token and the current user in local storage, and the route guards block unauthorized access. The scanner page, scan history page, and scan details page are only available after authentication, while public access is restricted to the login and registration pages.

The following figures are reserved for screenshots from the application:

Screenshot placeholder: login page

Figure 3.4: Login page

Screenshot placeholder: scanner page

Figure 3.5: Scanner page

Screenshot placeholder: scan history page

Figure 3.6: Scan history page

Screenshot placeholder: scan details page

Figure 3.7: Scan details page

This implementation also prepares the rest of the application by ensuring that scan creation and scan history remain protected, while guest users can only use limited public interactions.

3.6 Conclusion

This sprint established the authentication and access control layer of the HexStrike platform. It provided a secure login flow, a temporary guest mode, and protected access to sensitive pages and API endpoints.

The next sprint can now focus on the scanning workflow, vulnerability processing, and reporting features on top of this secured foundation.

Chapter 4

Scan Processing Pipeline & Normalization

4.1 Introduction

This sprint focuses on the scanning pipeline and on the vulnerability processing features of the HexStrike platform. The goal of this iteration is to implement reliable scan orchestration, normalize heterogeneous tool outputs into a unified schema, match findings against the vulnerability catalog, and persist results for later analysis and reporting.

In practice, this sprint covers scan submission, queueing and scheduling, worker orchestration, normalization, vulnerability matching, result persistence, and the user interfaces to submit and view scans.

4.1.1 Sprint Purpose

The purpose of this sprint is to deliver a robust scan processing pipeline that reliably executes scanners, normalizes their outputs, matches findings to known vulnerabilities, and stores results for retrieval and analysis. Success is measured by correct job processing, normalized result coverage for representative tool outputs, accurate matching to the catalog, and end-to-end retrieval from the UI.

4.2 Sprint Backlog

The backlog for this sprint contains the following user stories:

As a user, I can submit a scan target (URL) and receive a job identifier.
As a user, I can schedule or queue scans for later execution.
As the system, I can run scanner tools and collect raw outputs.
As the system, I can normalize raw outputs into a unified `ScanResult` schema.
As the system, I can match normalized findings to the vulnerability catalog and assign severities.
As a user, I can view, filter, and export scan history and results.
As an operator, I can monitor queue status and worker health.

Table 4.1: Sprint Backlog

4.3 Requirements Analysis

4.3.1 Use Case Diagram

The primary actors are the authenticated user (or guest if allowed), the API, and the scan worker. The use case diagram illustrates submission, processing, normalization, matching and viewing flows.

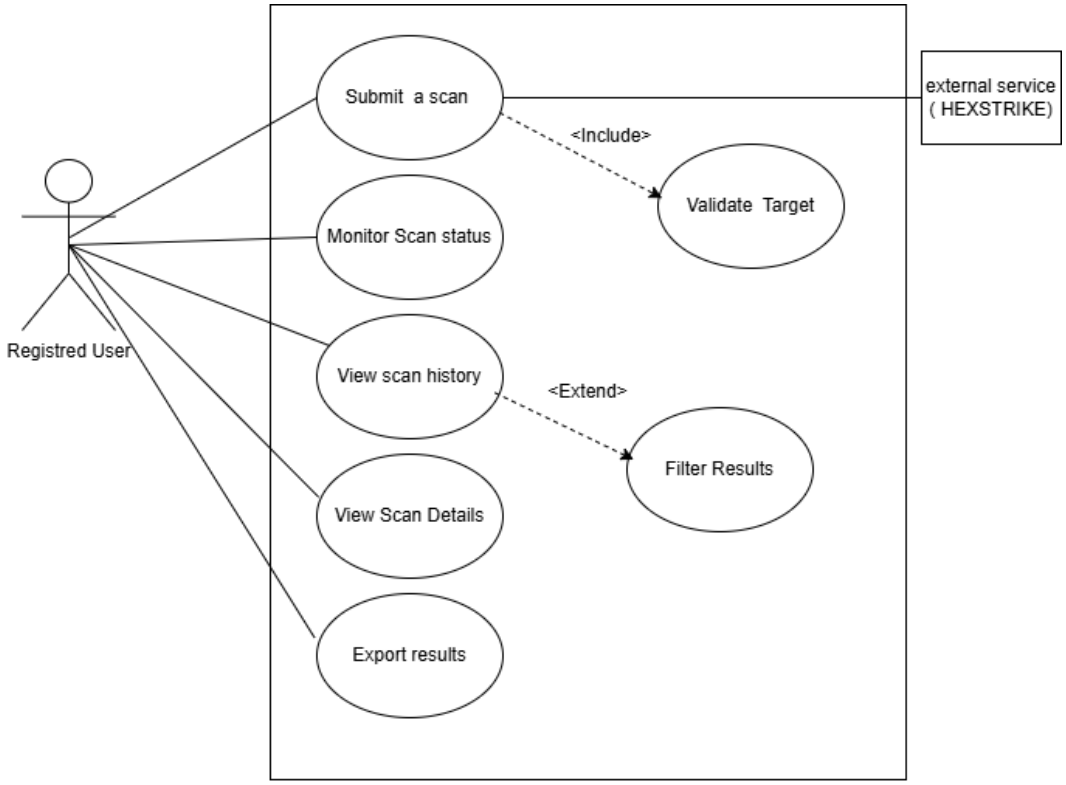

Figure 4.1: Use case diagram for scanning and vulnerability processing

4.3.2 Use Case Description

The main use cases of this sprint are summarized in the following tables.

Use case	Submit Scan
Actor	Authenticated user (or Guest if permitted)
Description	The user provides a target (URL) and optional parameters; the backend validates input, creates a `Scan` job, enqueues it and returns a job identifier.
Pre-condition	The user is authorized to submit scans and the queue service is available.
Post-condition	A `Scan` job is stored and queued for processing; the user receives the job id.

Table 4.2: Use case description: Submit Scan

Use case	Normalize Result
Actor	Scan Worker / Normalizer Service
Description	The worker parses raw tool outputs, maps fields to the unified `ScanResult` schema, and records structured findings.
Pre-condition	Raw output produced by a scanner is available on disk or in storage.
Post-condition	A normalized `ScanResult` object is produced and passed to the matcher and persistence layers.

Table 4.3: Use case description: Normalize Result

Use case	Match Vulnerability
Actor	Matcher Service
Description	The matcher takes normalized findings and attempts to correlate them with entries in the vulnerability catalog, assigning CVE identifiers or severity levels where applicable.
Pre-condition	Normalized findings are available; the vulnerability catalog is loaded and accessible.
Post-condition	Findings are annotated with matching vulnerability metadata and severity.

Table 4.4: Use case description: Match Vulnerability

Use case	View Scan History
Actor	User
Description	The user requests a list of past scans, applies filters, and inspects a scan's details and matched vulnerabilities.
Pre-condition	Scan results have been persisted and the user is authorized to view them.
Post-condition	The requested data is displayed; the user may export or navigate to details.

Table 4.5: Use case description: View Scan History

4.4 Design

4.4.1 Class Diagram

The principal components include the API, the queue, worker processes, the normalizer and matcher services, and the persistence layer. The class diagram will show the data models and service classes.

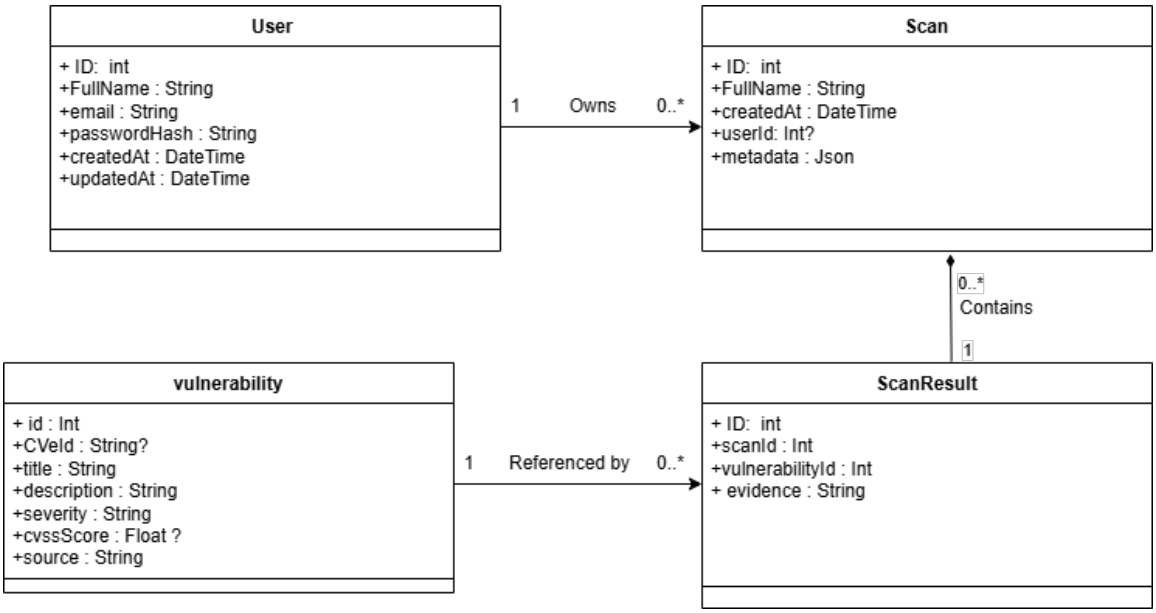

Figure 4.2: Class diagram for Sprint 2

4.4.2 Scan Workflow

The scan workflow describes the end-to-end processing of a submitted target: submission, queueing, worker execution of tools, normalization, matching, and persistence. The diagram below summarizes the sequence and components involved.

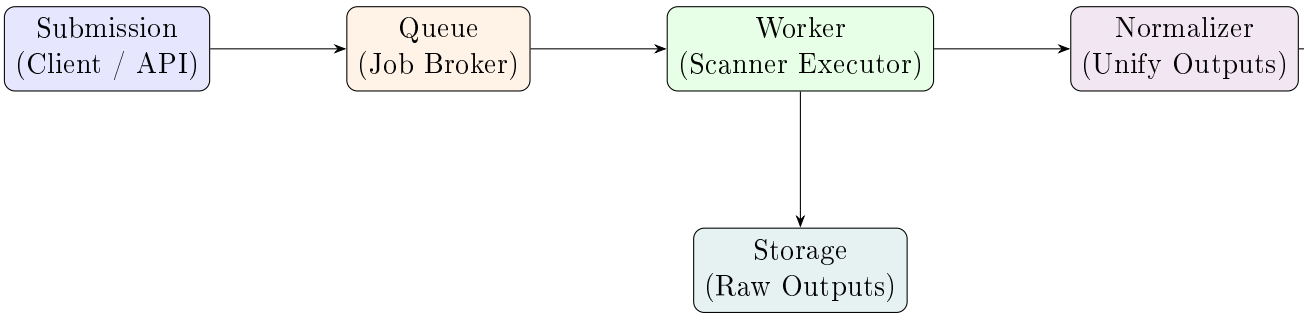

Figure 4.3: Scan workflow overview

4.4.3 Sequence Diagram

Key sequences include submission-to-persistence and normalization-to-matching. A sequence diagram will illustrate the interactions between client, API, queue, worker, normalizer, matcher and DB.

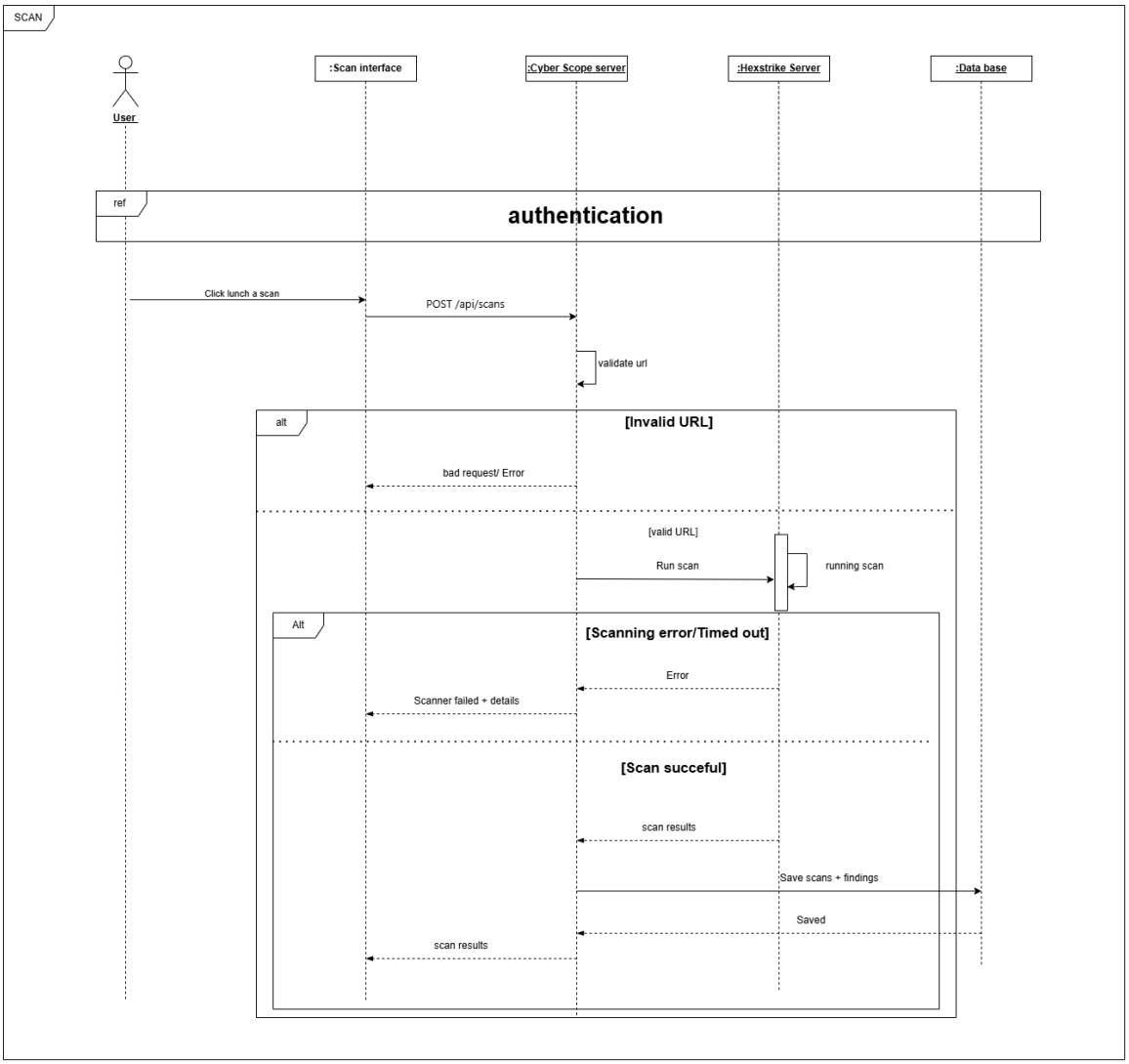

Figure 4.4: Sequence diagram for scan processing

4.5 Implementation

This sprint builds on the existing Node.js backend and React frontend and is illustrated with screenshots of the application interfaces.

Backend: new scan orchestration components were added (controller, service, worker/scheduler and normalization/matcher submodules). The API exposes endpoints to submit a scan, query job status, and retrieve results. Background workers consume queued jobs, run configured scanners, and store raw outputs for normalization and matching. Persistent domain data (scans, scan results, vulnerability catalog) is stored in the database.

Frontend: the UI was extended with pages and components to submit scans and inspect their results. The Scanner page allows users to start a scan and shows the returned job identifier. The Scan History page lists past scans with filters for date, target and severity. The Scan Detail view presents normalized findings and matched vulnerabilities with contextual information and links to remediation guidance.

Operational notes: workers and the queue are monitored for health and throughput; normalized results are validated before matching to the vulnerability catalog. Generated intermediate artifacts (raw scanner output) are stored temporarily for processing and auditing; normalized results and match annotations are persisted for reporting.

Figures reserved for this section: scanner submission UI, queue/worker dashboard, scan history, and scan detail views.

Screenshots from the application illustrating the features described in this chapter follow immediately after this section.

Screenshot placeholder: scanner submission page

Figure 4.5: Scanner submission page

Screenshot placeholder: queue / worker dashboard

Figure 4.6: Queue and worker dashboard

Screenshot placeholder: scan history page

Figure 4.7: Scan history page

Screenshot placeholder: scan details and matched vulnerabilities

Figure 4.8: Scan details page

4.6 Conclusion

This sprint builds the core scan processing pipeline and the infrastructure to normalize and match findings. With results persisted and accessible through the UI, future sprints can focus on reporting, enrichment, and advanced correlation features.

Chapter 5

Attack Graph Generation

5.1 Introduction

This sprint is dedicated to attack graph generation. The goal is to transform project artifacts into diagrams that show how data, endpoints and vulnerabilities are connected, so these diagrams highlight potential attack paths in the platform.

5.2 Sprint Purpose

The purpose of this sprint is to implement the Dashboard's attack-path and scenario generation features. The generator is an AI-assisted service that produces ephemeral attack scenarios (ordered steps) and a graph object (nodes and edges) for visualization in the Dashboard. Unlike static report diagrams, these graphs are runtime artifacts produced on-demand and returned to the frontend for interactive rendering.

5.3 Sprint Backlog

The backlog for this sprint focuses on the AI-driven scenario and risk-generation flows and the Dashboard UI integration:

As an analyst, I can request an attack scenario for a specific vulnerability and receive structured steps and a graph object.
As an analyst, I can request a risk assessment for a completed scan and receive a summarized JSON assessment.
As the system, I must validate and normalize AI responses into a safe, consistent graph payload (nodes/edges).
As a frontend, the Dashboard should render the returned graph (React Flow) and allow exporting the scenario as JSON.
As an operator, the service should handle AI timeouts, malformed responses, and return useful error messages.

Table 5.1: Sprint Backlog

5.4 Requirements Analysis

5.4.1 Generation Pipeline

The generator implements a compact, AI-assisted runtime pipeline: the Dashboard requests a scenario for a selected vulnerability; the backend loads the vulnerability context, builds a controlled prompt, calls the configured AI gateway, extracts and validates the returned JSON, maps normalized steps into an ephemeral graph (nodes

and edges), and returns the graph and ordered steps to the frontend for interactive rendering.

Placeholder: runtime pipeline (Vulnerability/Scan → AI gateway → parser → ephemeral graph)

Figure 5.1: Attack graph runtime pipeline (placeholder)

5.4.2 Use Case Diagram

Placeholder: Use case diagram (select vulnerability → generate scenario → inspect node telemetry)

Figure 5.2: Use case diagram for attack-path generation

5.5 Design

5.5.1 Graph Data Model

The attack graph is a simple, on-demand diagram that helps an analyst understand how an attacker could move through the system. It represents a generated scenario as a sequence of human-readable steps — each step has a short title, a clear description of the action, and a note about how serious it is. The diagram shows those steps as boxes connected by arrows to indicate possible flows or next actions. When an analyst clicks a box, the interface shows helpful details and suggested countermeasures for that step.

5.5.2 Sequence Diagram (runtime)

This sequence diagram summarizes the main runtime interaction for on-demand scenario generation: the Dashboard, the backend orchestration service, the AI gateway, and the frontend renderer. It visualizes the request flow (user selection → generation request → AI call → parsing/validation → interactive rendering) and highlights where failures (timeouts, malformed responses) are handled. The diagram emphasizes high-level handoffs rather than implementation details.

Placeholder: Sequence diagram (Dashboard → Controller → Service → AI → Parser → Dashboard)

Figure 5.3: Sequence diagram for runtime scenario generation

5.6 Implementation

The implementation focuses on a compact orchestration layer that connects the Dashboard to an AI-based scenario generator while keeping the UI interactive and user-friendly.

- Backend: a lightweight service accepts generation requests, enriches them with vulnerability context, calls the configured AI gateway, and validates and normalizes the returned scenario. The service converts the scenario into an ephemeral graph optimized for client rendering and returns it to the Dashboard. It also handles error conditions (missing records, invalid AI output, gateway timeouts) and surfaces clear HTTP status codes and messages for the frontend to display.

- Frontend: the Dashboard triggers scenario generation for a selected vulnerability, displays progress and error states, and renders the returned graph for interactive inspection. Selecting a node reveals tactical details and suggested countermeasures in the intelligence pane; inventory filters allow analysts to narrow focus before generation.

- Operational notes: generated graphs are ephemeral for this sprint and are not persisted. Raw AI responses are retained only when necessary for debugging and must be protected in production. AI gateway settings (endpoint, model, timeout) are configurable via environment variables and should be tuned for reliability and operator monitoring.

Figure 5.4: Dashboard attack graph (placeholder)

5.7 Conclusion

This sprint establishes the attack-graph generation workflow: extraction from source artifacts, enrichment with vulnerability metadata, and rendering to visual formats. The produced graphs will be used in later chapters to illustrate attack paths and support evaluation.

Chapter 6

HexStrike Internals and AI Integration

6.1 HexStrike Internals and AI Integration

This chapter focuses on HexStrike: its value, internal architecture, and operational details. HexStrike is the orchestration and automation engine that powers tool selection, execution, and intelligent decision making for the project. The chapter highlights how HexStrike complements the CyberScope pipeline, the internals of agent interaction, orchestration, data processing and the AI integration used for risk assessment.

6.2 Why HexStrike matters (Value Proposition)

HexStrike provides a centralized MCP-compatible orchestration layer that coordinates multiple security tools, optimizes parameters using AI, and aggregates results into actionable intelligence. Key benefits are summarized in Table 6.1 and visualized in Figure 6.1.

Table 6.1: Feature / Benefit summary

Feature	Problem solved	Impact
Automated orchestration	Eliminates manual coordination across tools and brittle scripts	Repeatable runs
AI-assisted selection	Reduces noisy results and manual parameter tuning	Higher precision
Worker scheduling	Mitigates resource contention across workers	Improved throughput
Caching	Avoids re-running identical scans	Lower cost

System component diagram (placeholder) – replace with real graphic in `uploads/diagrams/`

Figure 6.1: High-level component view (placeholder).

6.3 Architecture & Components

This section lists the main HexStrike components and responsibilities. See Table 6.2 for a compact component summary and Figure 6.2 for the system diagram.

Table 6.2: Component summary

Component	Responsibility	Interfaces / Outputs
MCP Server	Task distribution, agent registry	HTTP / gRPC
AI Decision Engine	Tool selection and parameter optimization	Decision API
Worker Manager	Spawn and schedule workers; health monitoring	Worker queue, health API
Cache	Result deduplication and TTL policies	Key-value store
Database	Persist scan results and queries	SQL / REST
UI Connector	Frontend API surface	REST / WebSocket

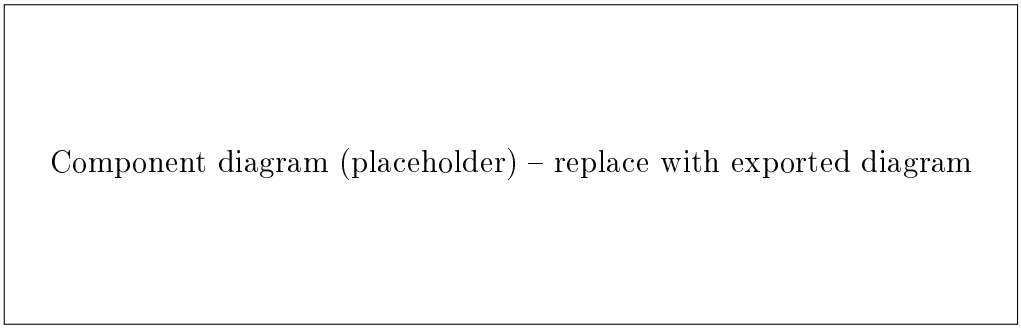

Figure 6.2: HexStrike component diagram (placeholder).

6.4 Agent Interaction & MCP Protocol

Agents connect to HexStrike using the MCP protocol. They register capabilities, receive tasks, and stream results back. A swimlane-style sequence (Agent / MCP / Worker / Tool) captures registration, task assignment, execution and streaming of results; include an illustrated diagram only if needed.

6.5 Tool Orchestration & Execution Flow

Tool orchestration covers discovery, selection by the decision engine, execution scheduling, retries and fallback strategies. The text summarizes decision points and parallel execution branches; include a flowchart only when a visual is required for reviewers.

6.6 Data Pipeline: Normalization, Matching & Enrichment

Raw scanner outputs are transformed into structured `ScanResult` objects, matched against a vulnerability catalog, and enriched with CVE and CVSS data. Table 6.3 gives a short schema fragment and Figure 6.3 shows the data flow.

Table 6.3: Key fields in `ScanResult` (example)

Field	Type	Description
id	string	Unique identifier for the finding
source	string	Tool that produced the finding
evidence	object	Raw evidence and context (headers, request/response)
severity	string	Normalized severity (low/medium/high)
references	array	CVE ids and external links

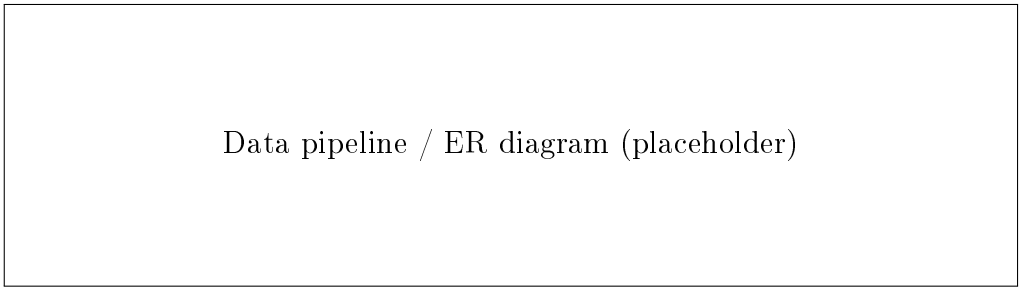

Figure 6.3: Data pipeline and entity relationships (placeholder).

6.7 Process Management, Caching & Error Recovery

HexStrike employs process-level state management for workers and a caching layer to avoid re-running expensive scans. Error recovery strategies include exponential backoff, automatic retries, and fallback scanning behaviors. Table 6.4 summarizes retry/backoff policies; include a worker state diagram only if visual clarification is needed.

Table 6.4: Retry/backoff policy (example)

Condition	Action	Notes
Transient network error	Retry up to 3 times	Exponential backoff: 2s, 4s, 8s
Tool timeout	Retry once with increased timeout	Log and escalate after failure
Permanent error	Mark failed	Store diagnostics for analyst

6.8 Conclusion

HexStrike acts as the project's orchestration backbone. The architecture and internal workflows presented in this chapter demonstrate how the system automates complex

multi-tool assessments and provides higher-level intelligence to reduce analyst workload.

References

[1] Ernst & Young, "Ey tunisia official website," 2026.

[2] OWASP Foundation, "Owasp top 10 - web application security risks," 2021.

[3] IBM, "Cost of a data breach report 2024," 2024.

[4] Verizon, "2024 data breach investigations report," 2024.

[5] CIRT, "Nikto web server scanner."

[6] OWASP, "Owasp zap."

[7] PortSwigger, "Burp suite."

[8] StackHawk, "Stackhawk dynamic application security testing."

[9] Detectify, "Detectify security platform."

[10] Snyk, "Snyk application security platform."

General Conclusion

Conclusion